Starting Simple: AI Tax

Design Decisions and Development Log

Project Team

May 2, 2026

Contents

1 Introduction **2**
- 1.1 Project Goals . . . 2
- 1.2 Non-Goals (v1) . . . 2
- 1.3 Phasing . . . 2

2 Philosophy **3**
- 2.1 Strong typing . . . 3
- 2.2 Compositional development . . . 3
- 2.3 Correct by construction . . . 3
- 2.4 Functional core, imperative shell . . . 4
- 2.5 Reproducibility and auditability . . . 4

3 Architectural Overview **4**

4 Phase 1: Self-Assessment Filing **5**
- 4.1 Goal . . . 5
- 4.2 Scope . . . 5
- 4.3 Architectural focus . . . 5
- 4.4 Success criterion . . . 5
- 4.5 Risks specific to P1 . . . 5

5 Phase 1: Component Design **6**
- 5.1 Pipeline at a glance . . . 6
- 5.2 Components . . . 6
  - 5.2.1 Ingestion . . . 6
  - 5.2.2 Fact Builder . . . 6
  - 5.2.3 Classifier . . . 7
  - 5.2.4 Rule Library . . . 7
  - 5.2.5 Tax Engine . . . 7
  - 5.2.6 Return Assembler . . . 7
  - 5.2.7 Explanation Renderer . . . 7
  - 5.2.8 Reconciliation . . . 8
- 5.3 Cross-cutting concerns . . . 8
  - 5.3.1 Provenance . . . 8
  - 5.3.2 Versioning . . . 8
  - 5.3.3 Persistence and dialogue . . . 8
- 5.4 What is not in the P1 core . . . 8

6 Phase 2: What-If Scenarios **8**
6.1 Goal . . . 8
6.2 Scope . . . 9
6.3 Architectural focus . . . 9
6.4 Success criterion . . . 9
6.5 Risks specific to P2 . . . 9

7 Phase 3a: Financial Planning Advice **9**
7.1 Goal . . . 9
7.2 Scope . . . 9
7.3 Architectural focus . . . 10
7.4 Success criterion . . . 10
7.5 Risks specific to 3a . . . 10

8 Phase 3b: Regulatory Compliance **11**
8.1 Goal . . . 11
8.2 What generalises . . . 11
8.3 What does not generalise . . . 12
8.4 Candidate regimes . . . 12
8.5 Architectural focus . . . 12
8.6 Success criterion . . . 13

9 Design Decisions **13**
9.1 DD-01: Trusted Inner Core in a Strongly-Typed Language (Haskell or Lean, Deferred) . . . 13
9.2 DD-02: Money Represented as Integer Minor Units . . . 13
9.3 DD-03: Phantom Types for Jurisdiction and Tax Year . . . 14
9.4 DD-04: Smart Constructors for All Domain Types . . . 14
9.5 DD-05: AI Outputs Never Bypass the Core . . . 14
9.6 DD-06: Property-Based Testing Required for All Tax Rules . . . 14
9.7 DD-07: Tax Computation as a Pure, Replayable Function . . . 15
9.8 DD-08: Scenarios as Composable Input Transformations . . . 15
9.9 DD-09: Time as an Explicit Dimension from Day One . . . 15
9.10 DD-10: Strict Separation of Computation, Guidance, and Advice . . . 15
9.11 DD-11: Monte Carlo with Seeded RNG for Projections . . . 16
9.12 DD-12: Python as Initial Implementation; Migrate Inward Over Time . . . 16
9.13 DD-13: Ratchet Discipline — Code Migrates Inward Only . . . 17
9.14 DD-14: Python ↔ Inner Core Boundary Protocol (Working Default) . . . 17
9.15 DD-15: Tax Computation Produces a Trace, Not a Number . . . 17
9.16 DD-16: Provenance Is a Threaded Value, Not a Sidecar . . . 18
9.17 DD-17: Recommendation Constructible Only Through the Suitability Gate . . . 18
9.18 DD-18: Python Is for the Ecosystem, Not for the Adapter Layer . . . 19

10 Open Questions **19**

11 Change Log **20**

A Module Reference **21**
A.1 Python Modules . . . 22
A.2 Inner-Core Modules (Haskell or Lean) . . . 22

1 Introduction

This document tracks the development of a platform that begins as a UK Self Assessment filing tool (Phase 1, §4), grows through tax-aware what-if optimisation (Phase 2, §6), and culminates in financial planning advice that competes with a human adviser (Phase 3a, §7), and an optional Phase 3b (§8) that generalises the architecture to regulatory compliance as a horizontal product. The system is implemented initially in Python and grows a strongly-typed inner core (in Haskell or Lean 4, see DD-01) as components stabilise. The end state is a thin Python outer shell wrapping a large, type-checked inner core. The system follows the *correct-by-construction* paradigm: invalid states should be unrepresentable in the type system of the inner core, and every public function in that core should be total and provably correct against its specification.

The AI component augments, rather than replaces, the deterministic core. The trusted computational kernel performs all tax arithmetic; the AI layer handles classification, extraction, and natural-language interaction — always with its outputs flowing back through the type-checked core for verification.

1.1 Project Goals

- Compute correct tax liabilities under a clearly specified jurisdiction's rules.
- Refuse, at compile time where possible, to produce ill-formed returns.
- Use AI to reduce manual data entry (receipt OCR, expense classification, question answering) without delegating arithmetic to it.
- Provide an audit trail mapping every figure on the return to its source.
- Showcase the power of the correct-by-construction approach: the construction of audits, reports, and explanations should be straightforward and rigorous, falling out of the architecture rather than being bolted on.
- Create the prototype for a business: the system is not a research artefact or a demo, but the technical foundation on which a commercial product can be built — whether a consumer financial-planning product (§7) or a horizontal regulatory-compliance platform (§8).

1.2 Non-Goals (v1)

- Replacing a qualified accountant for complex cases.
- Supporting every jurisdiction; we begin with UK Self Assessment and generalise.
- Filing returns directly with the tax authority; we generate the document, the user files it.

1.3 Phasing

Development is structured in three phases, each given its own section:

- Phase 1 (§4): Self-Assessment filing.
- Phase 2 (§6): what-if scenarios.
- Phase 3a (§7): financial planning advice (guidance product).
- Phase 3b (§8): regulatory compliance as a horizontal product, with FCA personal advice as the first concrete regime.

Architectural decisions in P1 are deliberately made to keep P2 cheap, and P3 is treated as a substantially different problem (time, uncertainty, and regulatory exposure) that builds on but does not re-use every component of P1/P2.

2 Philosophy

The system's design is governed by a small number of principles. They are not slogans; each one cashes out into specific decisions in this document.

2.1 Strong typing

Types are the cheapest correctness mechanism we have. A bug caught at compile time is one that never reaches a user, never requires a test, and never produces an audit-trail anomaly. We therefore reach for the strongest type system that is practical for the layer being built. The interface and adapter layers tolerate Python's gradual typing because their job is messy I/O; the trusted inner core demands a type system in which illegal states are unrepresentable (DD-01).

In practice: domain values cannot be constructed except through smart constructors that validate inputs (DD-04); jurisdiction and tax year are tracked at the type level so a UK 2024 figure cannot be added to a UK 2025 figure (DD-03); the result of an AI call is parsed into a typed value before it is allowed to influence any computation (DD-05).

Strong typing significantly reduces the attack surface — vital as security attacks become cheaper. Many vulnerabilities are bugs a sufficiently strong type system would have refused to compile: untrusted strings reaching SQL or shell interpreters, deserialisation flaws conjuring invalid objects past validation, confused-deputy bugs mixing authority levels the types could have kept apart. The smart-constructor and parse-don't-validate disciplines we adopt for correctness (DD-04, DD-05) are the same disciplines that close these doors. As AI lowers the cost of finding and exploiting weaknesses, the marginal value of every compile-time check rises.

2.2 Compositional development

Small pieces, well-typed, that compose. We prefer functions over methods, values over objects, and explicit composition over implicit framework magic. Where the domain has algebraic structure — combining tax inputs, layering scenarios, accumulating allowances — we expose that structure rather than hiding it behind imperative APIs.

The clearest payoff is in P2: scenarios are values of type `TaxInput` $\rightarrow$ `TaxInput` (DD-08), and combinations of scenarios are ordinary function composition. We get "extra pension AND marriage allowance transfer" for free, with no orchestration code, because we built the parts to compose.

2.3 Correct by construction

The phrase has two halves and we mean both.

Correct: the code does what it claims to do, verified by the strongest mechanism available — compile-time proofs in Lean if we go that way, types and property tests in Haskell, types and property tests and runtime assertions in Python.

By construction: correctness is not bolted on with after-the-fact validation. It comes from the way values are built. A `TaxYear` value is correct because there is no way to construct an invalid one; we do not need to remember to validate it before use, because it could not have been created any other way.

At its core, correct by construction is asking the developer to reflect in the program the specification they have in their head. The type system is the medium for that reflection: every invariant the developer believes holds about a value must be either expressed in its type or surrendered as an unchecked assumption. There is no third option. The discipline is what turns vague intent into machine-checkable fact.

2.4 Functional core, imperative shell

A discipline borrowed from Gary Bernhardt: the parts of the system that must be correct are pure functions on values; the parts that must talk to the world are kept thin and pushed to the edge. In our architecture (§3) the trusted inner core is the functional core, and the Python adapter and interface layers are the imperative shell. The shell prepares inputs and renders outputs; it does not contain rules. This is the philosophical statement of which DD-12 is the implementation.

2.5 Reproducibility and auditability

Any tax figure or recommendation the system produces must be reconstructible: given the same inputs, the system must produce identical outputs forever. Stochastic components are seeded (DD-11). Rule sets are values, parameterised by tax year, so we can replay an old computation under the rules that applied at the time. Each output figure is annotated with the inputs and rules that produced it.

The reason is not aesthetics. A recommendation made on Tuesday must survive a challenge on Friday — by the user, by HMRC, or by a compliance reviewer.

3 Architectural Overview

The system is architected as concentric layers. Outer layers handle interaction with the world (I/O, AI calls, UI); inner layers handle computation that must be trusted. Trust-critical code lives in a strongly-typed inner core; everything else lives in a Python shell.

From outside in:

1. **Interface Layer.** CLI, web frontend, API. Renders data, collects input, formats output for users. Predominantly Python in early phases; ultimately a thin shell.

2. **Adapter Layer.** I/O, AI provider calls, OCR, file parsing, persistence, external APIs. Validates everything before passing it inward. Predominantly Python in early phases because the AI/ML ecosystem (LLM SDKs, OCR, document-parsing libraries) is overwhelmingly Python-native. Pieces of the adapter that do not depend on that ecosystem — HMRC Making Tax Digital API clients, persistence, parsing of well-defined file formats — are legitimate inward-migration candidates. The criterion is ecosystem, not IO-ness: Haskell's `IO` is perfectly capable of any adapter work that does not need Python's libraries, and a typed adapter is in fact *stronger* at the trust boundary because there is no deserialisation step between adapter and core. See DD-18.

3. **Trusted Inner Core (Haskell or Lean, see DD-01).** Tax rules, money arithmetic, scenario composition, projections. Pure, total, type-checked. Begins small in P1 and grows over time as components are migrated inward (DD-12, DD-13).

The boundary between untrusted external data and the trusted inner core is the critical trust boundary: nothing crosses it without being parsed by a smart constructor in the inner core's language. That boundary may sit between Python and Haskell (in which case DD-14 governs the wire protocol) or, for pieces that have already migrated, entirely inside Haskell — in which case the smart-constructor discipline (DD-04) is enforced by ordinary function calls and DD-14 does not apply.

4 Phase 1: Self-Assessment Filing

4.1 Goal

A working tool that ingests a user's income and expense data and produces a correct UK Self Assessment return, with a clear audit trail showing how each figure was computed. The deliverable is a generated SA100 (with relevant supplementary pages) that the user files with HMRC, plus a per-line breakdown showing the rule and source data behind each figure.

4.2 Scope

In scope: employment income, self-employment, property income, dividends, savings interest, capital gains (straightforward cases), Gift Aid, pension contributions claimed at higher rate, marriage allowance.

Out of scope: trusts, non-domiciled status, foreign tax credits beyond simple cases, partnerships, complex CGT (share matching, EIS/SEIS, EMI options).

4.3 Architectural focus

P1 establishes the foundations the later phases depend on. Every architectural decision in this phase is chosen to keep P2 and P3 cheap.

- DD-02 (integer-pence money) and DD-03 (jurisdiction phantom types) define the domain types.
- DD-04 (smart constructors) governs how domain values enter the system.
- DD-05 (AI never bypasses the core) governs the trust boundary between the AI-driven adapter layer and the computation.
- DD-07 (pure replayable computation) is the architectural decision that makes P2 inexpensive: get this right in P1 and P2 is largely a ranking layer over an existing function.

The strategic posture from P1 onwards is to grow the inner core as far as possible — bounded by Python's irreplaceable ML/AI ecosystem (DD-18) — and as far as appropriate — bounded by component maturity and the ratchet (DD-13). This is not deferred migration. Anything stable, trust-critical, and not ecosystem-bound should land inside the typed core in P1, not in some imagined later phase. The risk being managed is not "too much Haskell too early" but its opposite: a Python codebase that ossifies around bad invariants and never migrates because each candidate is always "not quite ready". P1 starts small because P1 *is* small, not because the inner core is something we will get to later.

4.4 Success criterion

An independent test set of representative returns calculated to the penny against HMRC reference figures.

4.5 Risks specific to P1

- **Rule completeness.** HMRC's rules are large; a partial implementation that quietly misses a case will compute a confidently wrong number. Mitigation: explicit "unsupported case" types that fail loudly rather than guess.
- **Edge cases at allowance taper boundaries.** Property-based testing (DD-06) targeted at threshold values.

5 Phase 1: Component Design

This section decomposes Phase 1 into specific components, the data that flows between them, and the cross-cutting concerns that touch all of them. The decomposition is structured around the philosophical commitments in §2: components are pure where possible (functional core, imperative shell), the trust boundary is explicit (smart constructors at the entry to the inner core), and computation produces a trace from which explanation is derived (reproducibility and auditability).

5.1 Pipeline at a glance

1. **Documents and user input** flow into **Ingestion**, which produces `RawExtraction` values.
2. **Fact Builder** validates raw extractions and produces typed `Fact` values — the trust boundary into the inner core.
3. **Classifier** labels each fact with its tax category, producing `ClassifiedFact`.
4. **Tax Engine** applies the **Rule Library** to the classified facts, producing a `Computation` (a tree of rule applications, not just a final number; see DD-15).
5. **Return Assembler** reduces the computation to an `SA100` return.
6. **Explanation Renderer** reads the same computation tree to produce human-readable explanations of any figure.
7. **Reconciliation** runs alongside the pipeline, identifying missing or inconsistent data and generating questions for the user.

The pipeline is mostly linear, with Explanation reading the same intermediate artefact that Return Assembler reduces. This is deliberate: explanation is not a parallel narrative produced separately; it is a different view of the same evidence.

5.2 Components

5.2.1 Ingestion

Purpose. Extract structured data from heterogeneous sources — PDFs of P60s and P11Ds, receipts, bank-statement CSVs, user-typed forms.

Layer. Adapter (Python). AI-heavy: OCR, document classification, field extraction.

Output. `RawExtraction`: a value carrying the extracted text or number, a source-document reference, byte offsets within that source, and a confidence score.

Trust. None. Outputs of this stage are untrusted; downstream stages must validate.

5.2.2 Fact Builder

Purpose. Convert untrusted `RawExtraction` values into typed, validated `Fact` values that the rest of the system can trust.

Layer. Initially Python with Pydantic for runtime validation; this is an early candidate for migration into the inner core, where smart constructors (DD-04) provide compile-time guarantees rather than runtime checks.

Output. `Fact`: a closed sum type covering every kind of fact the system understands (employment income, dividend received, Gift Aid donation, etc.). Each variant carries its own validated fields and a `Provenance` reference (DD-16).

Trust boundary. This is the boundary between the imperative shell and the functional core (§2, DD-05). Anything that has type `Fact` has been validated; downstream code does not re-check.

5.2.3 Classifier

Purpose. Map each `Fact` to one or more tax categories — e.g. a bank-statement credit might be employment income, a dividend, or a gift, and the classification determines which rules apply.

Layer. Hybrid. The proposal of a category is AI-assisted; the acceptance of that proposal is type-checked. The classifier may also run deterministic rules (e.g. "income from employer X is always employment income, the user has confirmed this") ahead of any AI call.

Output. `ClassifiedFact`, comprising a `Fact`, a `Category`, and a `ClassificationProvenance`. Where the classifier is uncertain, the output is a list of candidates with confidences, deferred for user confirmation rather than guessed silently.

5.2.4 Rule Library

Purpose. Encode the fragment of UK Self Assessment rules in scope for P1 (§4). Each rule is a value with: an identifier, an applicability predicate, a computation, and a reference to the HMRC publication it codifies.

Layer. Inner core. This is one of the most important migration targets: rules-as-values rather than rules-as-code-paths is what makes DD-07 and DD-09 possible, and is also what lets the rule set itself be audited by a tax adviser independently of the engine that runs it.

Output. A `Rules y` value parameterised by tax year. The library is loaded from a versioned source. The format of that source — hand-written code, a declarative DSL, or YAML curated by a tax adviser — is open (see Open Questions).

5.2.5 Tax Engine

Purpose. Apply a `Rules y` value to a set of `ClassifiedFact` values, producing a tax computation.

Layer. Inner core. Pure function: same inputs always produce same outputs (DD-07).

Output. `Computation`: a directed acyclic graph of `RuleApplication` nodes. Each node records which rule fired, on which facts, producing which intermediate value, with which justification. The final tax liability is the value at the root of this graph; every intermediate figure is also addressable.

Critical property. The `Computation` is the canonical source of truth. Every other downstream artefact (the SA100 return, the explanation, the audit trail) is a view of this graph. See DD-15.

5.2.6 Return Assembler

Purpose. Reduce a `Computation` to an `SA100` value populated with the figures HMRC expects in each box.

Layer. Inner core. Pure.

Output. `SA100`: a typed structure mirroring the SA100 form, with each figure carrying a reference to the node in the `Computation` that produced it.

5.2.7 Explanation Renderer

Purpose. Produce human-readable explanations of any figure in the return: *"why is box 1 £42,500?"*.

Layer. Hybrid. The explanation *tree* is purely structural — it is a traversal of the `Computation` starting from a chosen node. The *prose* for each step may be AI-rendered for fluency, but the content is fixed by the rule applied; each `Rule` in the library ships with a canonical explanation template that the AI may elaborate but not contradict (DD-05).

Output. `Explanation`: a tree of statements, each referencing the `RuleApplication` it explains and the input values that justified it.

5.2.8 Reconciliation

Purpose. Identify what is missing, what is inconsistent, and what the user must clarify before a return can be filed.

Layer. Inner core (the logic) plus adapter (the presentation of questions to the user).

Output. A list of `Question` values, each with a category (missing data, ambiguous classification, inconsistency between sources) and a structured prompt suitable for either UI rendering or AI-mediated dialogue.

5.3 Cross-cutting concerns

5.3.1 Provenance

Every `Fact`, `ClassifiedFact`, and node in a `Computation` carries a `Provenance` reference. Provenance is not a sidecar log; it is a value threaded through the types so that, given any output number, the chain of inputs and rules behind it is reachable by traversing the computation graph (DD-16).

5.3.2 Versioning

The `Rules y` value carries its tax year and a content hash. The `Computation` embeds that hash so a return can be re-derived under the rules in force when it was generated, even years later.

5.3.3 Persistence and dialogue

The user works on a return across multiple sittings. State (the current `Fact` set, the `ClassifiedFact` set, outstanding `Question` values) is persisted between sessions. Persistence and the dialogue manager live in the adapter layer; the inner core remains stateless — it is given the full state on each invocation and returns the next state.

5.4 What is not in the P1 core

To keep the P1 surface tight, the following are deliberately deferred:

- Direct filing with HMRC's Making Tax Digital APIs (the user files the generated return manually).
- Multi-user or accountant collaboration features.
- Encrypted document storage beyond platform defaults (added when the customer profile demands it).
- Multi-jurisdiction returns (DD-03 makes this a future extension, not a P1 feature).

6 Phase 2: What-If Scenarios

6.1 Goal

On top of P1, the user can ask questions of the form *"what happens if I pay an extra £200/month into my pension?"* or *"what if I move £20,000 from a GIA into an ISA?"*. The system enumerates the tax consequences and ranks scenarios by net benefit, with reasoning shown.

6.2 Scope

In scope: pension contributions (relief at source vs net pay vs salary sacrifice), ISA vs GIA placement, marriage allowance transfer, Gift Aid timing, CGT harvesting against the annual exempt amount, dividend allowance optimisation.

Out of scope: multi-year scenarios, scenarios that depend on future returns, anything that requires modelling time or uncertainty (those belong to P3a).

6.3 Architectural focus

P2 is cheap to build only if the P1 architecture was right. The two decisions that pay off here are DD-07 (computation as a pure replayable function) and DD-08 (scenarios as composable input transformations). Together they make P2 roughly: enumerate scenarios, apply each to the baseline input, run the P1 computation, rank by net benefit.

If P1 was sloppy — if the tax computation has hidden state, depends on mutable globals, or is expensive in ways we did not audit — P2 reveals it. P2 is therefore also a stress test of the P1 implementation.

The scenario engine itself is a strong candidate for migration into the typed inner core (DD-12, DD-13). Scenarios are small, pure, and benefit from type-level checking: we want it to be impossible for an "add an ISA contribution" scenario to produce an invalid `TaxInput`.

6.4 Success criterion

On a panel of test cases curated with a chartered tax adviser, the system's ranked recommendations match the adviser's, with the reasoning visible and checkable.

6.5 Risks specific to P2

- **Combinatorial explosion.** N independent scenarios produce 2^N combinations. We need a sensible default presentation — probably top-K by net benefit, with a UI for exploring combinations on demand.
- **Interacting scenarios.** Some scenarios are not independent (salary sacrifice changes both gross pay and pension input). The composable-transformation design (DD-08) handles this correctly only if the underlying `TaxInput` is rich enough to represent the interactions; a too-simple input model will silently lose information.

7 Phase 3a: Financial Planning Advice

7.1 Goal

Multi-year projection and planning: retirement adequacy, inheritance-tax exposure, the tax-efficient ordering of withdrawals across pension / ISA / GIA in decumulation, lifetime-gifting strategy. The pitch is that the system can do, in 10 minutes of user time, what a financial planner charging £150/hour would do in five hours.

3a stays on the guidance side of the FCA perimeter (DD-10): the system ranks computed outcomes, explains them, and lets the user decide. It does not emit `Recommendation` values. That is the line that §8 optionally crosses.

7.2 Scope

In scope: 30+ year projections, Monte Carlo over investment returns and inflation, drawdown ordering strategies, IHT modelling, basic estate-planning illustrations.

Out of scope (v1 of 3a): insurance product recommendations, individual security selection, alternative-investment exposure, anything that touches the client's income protection or estate-planning legal documents directly. Personal recommendations of any kind — those belong to 3b, gated by FCA authorisation.

7.3 Architectural focus

3a is materially harder than P1/P2. Three things change:

1. **Time becomes a dimension.** DD-09 anticipated this; here it becomes load-bearing. The `Timeline` type is fully populated, and the rules data structure (DD-07) is parameterised by tax year so we can project forward under either current or scenario rules.
2. **Uncertainty becomes a dimension.** DD-11 (seeded Monte Carlo) is activated. Any output the system produces is a distribution, not a point estimate, and the UI must communicate this honestly.
3. **Regulatory exposure rises sharply.** DD-10 (computation / guidance / advice separation) becomes the most important decision in the document. A piece of code that emits a `Recommendation` value is knowingly approaching the FCA's regulated perimeter; the system must make that approach explicit and auditable, or refuse to make it at all.

By 3a, most numerical and logical computation should live in the typed inner core (DD-12). Python remains as a thin shell handling I/O, AI, and UI. The ratchet (DD-13) ensures we got there gradually.

7.4 Success criterion

On a panel of representative client situations, an independent panel of CFP-qualified reviewers judges the system's output to be at least as good as that produced by a typical £150/hour adviser, achieved in under 10 minutes of user time — with the explicit caveat that the system provides decision support, not a personal recommendation.

7.5 Risks specific to 3a

- **Regulatory perimeter.** See DD-10. The product must stay clearly on the guidance side. If it drifts, the firm has been giving unauthorised regulated advice, which is an existential risk. The discipline is in DD-10 (typed separation) and the gate is in DD-17; crossing the perimeter on purpose is the subject of §8.
- **Compelling without recommending.** "Here are ranked options, you decide" is a harder product to make compelling than "you should do X". Decision-support UX is a real design problem, not a compliance fig leaf.
- **Modelling assumption defensibility.** Return distributions, inflation models, and longevity curves are all judgement calls. They must be documented, sourced, and challengeable by the user.
- **Recommendation latency.** Monte Carlo runs over 30 years across thousands of paths may exceed our budget on the subprocess-and-JSON boundary (DD-14). The latency reckoning may force the FFI migration here.

8 Phase 3b: Regulatory Compliance

8.1 Goal

The architecture and team capabilities developed in P1 and P2 are, by alignment of incentives rather than by accident, almost exactly what every regulatory-compliance product needs. P3b is the phase in which that observation becomes the business: extend the platform from compliance with one regulator (HMRC) to a horizontal regulatory-compliance product capable of serving any regime that demands rules, evidence, audit, and retention.

The strategic claim is direct. Regulatory compliance is a multi-billion pound market in every developed economy — spend that mid-size firms and above already make and would prefer to make less of. The firms that win in that market are the ones with horizontally reusable infrastructure, because the per-regime work is repeatable but the kernel is not. P1 and P2 were never just "a tax product"; they were the cheapest possible forcing function for building that kernel against a real regulator with real penalties for being wrong. If the kernel survived HMRC, it survives most regulators.

P3b is not a continuation of P3a's consumer financial-planning product. The two are different businesses with different customers, sales motions, and unit economics. 3a is B2C, software margins, no authorisation cost unless deliberately taken on. 3b is B2B (or B2B2C), a horizontal compliance platform sold to firms that already pay for compliance, with each new regime a repeatable instance of the same architecture. Whether the firm pursues 3a, 3b, both, or neither is the strategic decision flagged in §10.

8.2 What generalises

The reusable kernel from P1/P2:

- **Trust boundary with smart constructors** (DD-04, DD-05). Every regulator's first demand is "how do you know the data is valid?". Smart constructors are the answer.
- **Rules as versioned, time-stamped data** (DD-07). Every regulator's rules change over time, vary by jurisdiction, and must be applied as they stood when an event occurred. Rules-as-data serves all of this.
- **Trace-as-output computation** (DD-15). Every regulator wants to see "which rule fired, on which inputs, producing which output". `Computation` graphs are exactly this artefact.
- **Threaded provenance** (DD-16). Every investigation follows a value back to its sources. A graph traversal does this in constant code.
- **AI-mediated extraction with typed verification** (DD-05). Every regime that processes documents (KYC paperwork, suitability records, ESG disclosures, AML SARs) needs the extract-then-verify pattern.
- **Reproducibility and audit** (§2). Seeded RNG, immutable rule sets, content-hashed inputs — the audit trail required by essentially every regulator.
- **Regulated outputs through lawfulness gates**. DD-10 and DD-17 are the FCA-specific instance of a more general pattern: outputs that the regulator restricts are typed such that they cannot be constructed except by passing through their lawfulness check. The pattern transfers directly: AML SARs through a triggering-event gate, GDPR data exports through a lawful-basis gate, ESG disclosures through a methodology gate.

8.3 What does not generalise

The per-regime surface (repeatable, but real engineering effort each time):

- The vocabulary — `Fact` subtypes specific to that regime.
- The rule library and its sources of authority.
- The output forms or reports the regulator demands.
- The retention policy and access controls.
- Classifier prompts and category taxonomies.
- Domain expertise. The architecture amortises engineering across regimes; it does not amortise legal and regulatory expertise. Each new regime needs an in-house or contracted expert.

8.4 Candidate regimes

In rough order of architectural fit and known commercial demand:

- **FCA personal financial advice** (the original 3b framing). Concrete instance of the lawfulness-gate pattern via DD-17 — the architectural work for this regime drove that decision.
- **AML / KYC** (UK MLR 2017, EU AMLD): customer due diligence, source-of-funds checks, suspicious-activity reporting. Heavy AI-extraction component (passport, utility bill, beneficial-ownership documents) on which DD-05 is exactly the right shape.
- **Data protection** (UK GDPR, EU GDPR): data inventory, lawful-basis tracking, subject-access requests, breach impact assessments, DPIAs. Threaded provenance (DD-16) addresses much of GDPR's audit requirement directly.
- **ESG and sustainability reporting** (SECR, TCFD, CSRD): extraction-and-aggregation heavy, computation light. Rule-as-data (DD-07) handles per-jurisdiction reporting differences without forking the engine.
- **FATCA / CRS** (cross-border tax reporting for financial institutions): residence determination plus reporting. Reuses much of P1's tax-domain vocabulary; commercially attractive because it is a near-duplicate of work the kernel already does.
- **Electronic-records compliance** (FDA 21 CFR Part 11, GxP, MHRA equivalents): records integrity, attribution, contemporaneity. DD-15 plus DD-16 satisfy the substantive requirements almost verbatim.
- **Consumer Duty** (FCA): outcomes-based good-customer-outcomes evidence. Trace and provenance are how a firm proves its outcomes are good.

8.5 Architectural focus

The 3b engineering effort is in (a) generalising the parts of P1/P2 that are HMRC-specific to be regime-pluggable, and (b) building the regime-specific surfaces for each new market entered. The kernel itself does not change.

The discipline of generalisation is to do it *when the second regime arrives*, not before. Refactoring HMRC-specific code into a "regime-pluggable" shape in advance of any second regime risks producing the wrong abstraction. The right time to design the regime interface is when there are at least two concrete regimes whose differences inform it.

8.6 Success criterion

- A second regulatory regime is implemented end-to-end on the same kernel, with no kernel-level changes required.
- The marginal engineering cost of that second regime is materially smaller than the cost of building HMRC tax was, with the delta attributable to the kernel.
- An external compliance reviewer for the second regime audits the system favourably against that regime's standards.
- At least one paying customer in each regime that has been productised.

9 Design Decisions

Each design decision is numbered (DD-NN), dated, and includes the alternatives considered and the rationale.

9.1 DD-01: Trusted Inner Core in a Strongly-Typed Language (Haskell or Lean, Deferred)

Date: May 2, 2026
Decision: The trusted inner core (introduced in DD-12) will be implemented in either Haskell (GHC 9.6+) or Lean 4. The choice is deferred until a representative slice of the core has been prototyped in both. Python is excluded for the inner core: its type system, even with `mypy --strict`, cannot prevent runtime construction of invalid domain values and cannot express the type-level invariants we rely on (DD-03, DD-04).
Alternatives considered: Rust (strong types but weaker type-level reasoning); OCaml (capable but smaller ecosystem); F# (would couple us to .NET); Idris (ecosystem too thin); staying in Python with stricter linting (rejected — the discipline is voluntary and breaks under deadline pressure).
Rationale (Haskell): mature, proven in production financial systems, expressive type system (GADTs, type families, phantom types), large ecosystem, hireable. Strong types but not proofs.
Rationale (Lean): first-class theorem proving — we can *prove* that, e.g., the tax-band function is monotonic in income within each band, rather than test it. If "provably correct tax computation" is a competitive differentiator for the product (especially for P3, where regulators and clients will scrutinise the model), Lean wins. Lean 4's tooling for general programming has matured significantly.
Decision criterion for resolving the deferral: prototype a small but representative slice (the `Money` type, two tax-band functions, one property such as monotonicity) in both. Choose based on: (a) effort to express the domain; (b) effort to state and discharge the chosen property; (c) realistic hiring pipeline.
Trade-offs: keeping the choice open costs us some specificity in early documentation. The cost of choosing wrong is much higher.

9.2 DD-02: Money Represented as Integer Minor Units

Date: May 2, 2026
Decision: All monetary amounts are stored as a 64-bit signed integer representing the smallest currency unit (pence for GBP). Wrapped in a dedicated nominal type — a `Money` class in Python, a `newtype` in Haskell, or a structure in Lean — so it cannot be silently mixed with raw integers.
Alternatives considered: `Decimal` (Python's `decimal` module / `Data.Decimal`); rationals; floating point.

Rationale: Floating point is excluded categorically: tax law rounds at specific points and floating-point error is unacceptable in a regulated context. Integer pence makes every operation exact and HMRC's rounding rules trivially expressible. Rationals risk unbounded denominator growth; `Decimal` adds dependency surface for no gain over a 64-bit integer in this domain.
Trade-offs: Percentage calculations and currency conversions must explicitly choose a rounding mode. This is a feature, not a bug.
Migration note: `Money` is a strong candidate to be the first module migrated into the inner core (DD-12).

9.3 DD-03: Phantom Types for Jurisdiction and Tax Year

Date: May 2, 2026
Decision: Tax-rule-bearing types are parameterised by phantom types representing jurisdiction and tax year, e.g. `Allowance UK Y2025`.
Alternatives considered: Runtime jurisdiction tag; separate modules per jurisdiction without a type-level marker.
Rationale: Phantom types make it a compile-time error to apply UK rules to US figures, or to mix tax years. The runtime cost is zero — phantoms are erased — and the safety gain is large.
Trade-offs: Slightly more verbose signatures.

9.4 DD-04: Smart Constructors for All Domain Types

Date: May 2, 2026
Decision: Domain types (`NIN`, `UTR`, `NonNegativeMoney`, `TaxYear`) export only smart constructors that validate inputs. The data constructors themselves are not exported.
Alternatives considered: Validation-on-use; LiquidHaskell refinements.
Rationale: Once a value has the type, downstream code can trust it. This is the foundation of the correct-by-construction approach. LiquidHaskell is interesting but the team is not yet fluent in it; we may revisit.
Trade-offs: More boilerplate; partial constructors return `Either ValidationError T`.

9.5 DD-05: AI Outputs Never Bypass the Core

Date: May 2, 2026
Decision: Any value produced by an AI model (classification, extracted figure, generated explanation) must be parsed by a smart constructor before being used in any calculation.
Alternatives considered: Trusting structured outputs from the model.
Rationale: Models hallucinate. The whole point of the trusted core is that we don't have to trust the AI; we re-validate everything it produces. Each AI-derived value also carries provenance metadata so any tax line can be traced back to its source document.
Trade-offs: Some latency from the parse-and-validate step; unparseable AI outputs require user intervention.

9.6 DD-06: Property-Based Testing Required for All Tax Rules

Date: May 2, 2026
Decision: Every tax rule must have property-based tests (QuickCheck / Hedgehog) covering monotonicity, threshold behaviour, and boundary conditions.
Rationale: Tax law is full of cliff edges and tapered allowances. Examples-based tests miss boundary bugs. Properties like "increasing income never decreases tax owed within a band" catch whole classes of error.
Trade-offs: Up-front cost in writing well-shrunk generators.

9.7 DD-07: Tax Computation as a Pure, Replayable Function

Date: May 2, 2026
Phase relevance: Foundational for P2; required for P3.
Decision: The core tax computation has the signature `computeReturn :: Rules y -> TaxInput y -> TaxReturn y`, with no `IO`, no hidden state, and no reference to global configuration. The `Rules` value is itself data — the rule set for a given tax year is a value, not code paths.
Alternatives considered: Effectful computation in a custom monad; hard-coding rules into the function body.
Rationale: P2's what-if scenarios require running the computation many times under perturbed inputs. If the computation is a pure function we get this for free; if it is effectful or stateful we will pay for the mistake every day of P2 and P3. Treating `Rules` as data also makes historical tax-year computations (re-running 2022/23 for an amended return or for projections) trivially expressible.
Trade-offs: The `Rules` type must be expressive enough to encode the rule set declaratively, which is a non-trivial design exercise.

9.8 DD-08: Scenarios as Composable Input Transformations

Date: May 2, 2026
Phase relevance: Defines the P2 abstraction.
Decision: A scenario is a value of type `Scenario y = TaxInput y -> TaxInput y`, equipped with a label and a provenance reason. Scenarios compose by ordinary function composition. The P2 engine takes a baseline `TaxInput`, a list of scenarios, and returns a ranked table of `(Scenario, TaxReturn, NetBenefit)`.
Alternatives considered: Encoding what-ifs as parameters on the calculation; embedding them as branches inside the rules.
Rationale: Modelling scenarios as input transformations keeps the trusted core untouched — a scenario cannot break a tax rule because it operates only on inputs, which the core then validates and computes against in the usual way. Composability gives us combinations (*"extra pension AND marriage allowance transfer"*) for free. Provenance metadata makes each recommendation auditable: we can show the user exactly which input changed and by how much.
Trade-offs: Some scenarios that interact non-locally (e.g. salary-sacrifice that changes both gross pay and pension input) need a slightly richer input model than the most naive design.

9.9 DD-09: Time as an Explicit Dimension from Day One

Date: May 2, 2026
Phase relevance: Light cost in P1 to enable P3a without rewrite.
Decision: Even in P1, the input and output types are tagged with the tax year (DD-03), all monetary fields are dated, and there is a distinguished `Timeline a` type representing a value that varies over tax years. P1 will use `Timeline` only trivially; P3a will populate it with projections.
Alternatives considered: Adding the time dimension only when P3a work begins.
Rationale: Retrofitting time into a type-level model after the fact is exactly the kind of pervasive change that breaks every signature in the codebase. The cost of including it from the start is small; the cost of adding it later is enormous.
Trade-offs: Slightly heavier types in P1 for benefit not realised until P3a.

9.10 DD-10: Strict Separation of Computation, Guidance, and Advice

Date: May 2, 2026
Phase relevance: Critical for both P3a (defines the product surface that stays on the guidance side) and P3b (defines what 3b lifts and what it does not); sets policy for the whole project.

Decision: The system distinguishes three categories of output, and each is implemented as a different type so they cannot be confused at the boundary:

- **Computation** (`TaxReturn`, `NetBenefit`): deterministic results of applying public rules to user data. Always produced.
- **Guidance** (`Guidance`): factual statements about how the rules work in general (*"higher-rate pension contributions receive 40% relief, of which 20% is reclaimed via Self Assessment"*). Always permitted.
- **Advice** (`Recommendation`): a personal recommendation to act (*"you should move £X from your GIA to your ISA"*). In the UK this falls within the FCA's regulated activities (RAO Article 53). The system will, by default, not produce `Recommendation` values; it produces ranked `Computation` results and lets the user choose.

Alternatives considered: Treat all output uniformly and rely on disclaimers; pursue FCA authorisation early.
Rationale: The line between guidance and regulated advice is the single largest risk to P3a. Encoding the distinction in the type system means a piece of code that wants to emit a recommendation must do so explicitly, making the regulatory review tractable. Whether the project ever crosses into the `Recommendation` space is the strategic question that defines whether P3b happens at all (§8); it is to be answered before P3a ships, not a technical one.
Trade-offs: The product surface for P3a is constrained until the business question is resolved. This is the correct trade-off.

9.11 DD-11: Monte Carlo with Seeded RNG for Projections

Date: May 2, 2026
Phase relevance: P3a.
Decision: Long-horizon projections (returns, inflation, longevity) are modelled by Monte Carlo simulation with an explicit, seeded random generator passed as data. A given input plus seed always produces the same output.
Rationale: Reproducibility is non-negotiable for an audit trail: a recommendation made on Tuesday must be reconstructible on Friday. Closed-form approximations of multi-asset, tax-aware drawdown problems do not exist; simulation is the standard tool.
Trade-offs: Compute cost; the need to choose return distributions defensibly (this becomes a documented modelling assumption).

9.12 DD-12: Python as Initial Implementation; Migrate Inward Over Time

Date: May 2, 2026
Phase relevance: Cross-cutting; defines the implementation strategy.
Decision: The system is implemented initially in Python (3.12+, `mypy --strict`, `pydantic` for runtime validation at the edges). As components mature and prove their value, the trust-critical ones — the `Money` type, tax-rule evaluation, the `Rules` data structure, scenario composition (DD-08), the projection engine — are migrated into the strongly-typed inner core (DD-01). The end state is a Python shell only where Python's ecosystem is irreplaceable (AI/ML, OCR, document parsing), wrapping a large typed core that owns all computation and any adapter work that does not depend on the Python AI/ML ecosystem. The end-state split is defined by ecosystem, not by layer; see DD-18.
Alternatives considered: (a) Start in Haskell/Lean from day one; (b) stay in Python forever with discipline.
Rationale: Python lets us iterate on product, model the domain, and explore AI integrations quickly. Starting in Haskell or Lean would impose a heavy cognitive load before we know the

shape of the problem. But Python's type discipline is too weak for the trust requirements at scale — runtime type-checking with `pydantic` is not a substitute for compile-time guarantees on a tax engine. We therefore begin migrating from day one, not merely plan for it. The strategic posture is to grow the inner core as far as is possible (DD-18) and appropriate (DD-13) at every phase, P1 included. "Wait for a later phase" is not a justification.
Trade-offs: Two languages and a boundary to maintain (DD-14). The migration must be disciplined (DD-13) or we end up with two half-systems and no benefit.

9.13 DD-13: Ratchet Discipline — Code Migrates Inward Only

Date: May 2, 2026
Phase relevance: Cross-cutting; protects the value of DD-12.
Decision: Once a module has been migrated from Python into the typed inner core, it does not migrate back. Bugs in migrated code are fixed in the inner core; new features that touch a migrated area are implemented in the inner core. There is no "temporary patch in Python"; if the inner-core implementation isn't ready, the feature waits.
Alternatives considered: Allowing pragmatic round-trips when delivery pressure demands it.
Rationale: The migration is costly to perform. Allowing code to slosh between languages defeats the purpose: we lose the trust guarantees of the inner core whenever a feature is "temporarily" patched in Python, and the patch becomes permanent. The ratchet rule converts the strategy into actual progress rather than language churn. The cost — occasional features delayed — is the cost of meaning what we said in DD-12.
Trade-offs: Sometimes a feature that touches a migrated module will have to wait for its inner-core implementation. This is a feature, not a bug — it forces honest sequencing of work.

9.14 DD-14: Python ↔ Inner Core Boundary Protocol (Working Default)

Date: May 2, 2026
Phase relevance: Activated as soon as the first inner-core module exists.
Decision (working default): Subprocess + JSON over stdio, with a schema validated on both sides — Pydantic on the Python side, the inner core's parser on the typed side. The inner core treats incoming JSON as untrusted and parses it through smart constructors (DD-04, DD-05). Output is the same in reverse.
Alternatives considered: (a) FFI directly into the inner-core language (faster, but tightly platform-coupled); (b) gRPC or HTTP over a local socket (more operationally complex, generalises to a service split); (c) shared-memory IPC (premature).
Rationale: The boundary defines the trust contract; whatever protocol we choose, both sides must validate. Subprocess + JSON is the simplest thing that works, gives clean process isolation, and makes the inner core trivially swappable between Haskell and Lean during the DD-01 prototype phase. We expect to migrate to FFI once latency on hot paths matters.
Trade-offs: Latency. Acceptable for P1 (one return per session) and most of P2 (tens to hundreds of scenario evaluations). Will need re-evaluation in P3a where Monte Carlo runs may need thousands of inner-core calls per session.

9.15 DD-15: Tax Computation Produces a Trace, Not a Number

Date: May 2, 2026
Phase relevance: Defines the P1 component design (§5); load-bearing for explanation, audit, and the P2 scenario engine.
Decision: The Tax Engine produces a `Computation` value — a directed acyclic graph of `RuleApplication` nodes, each recording which rule fired, on which inputs, producing which intermediate value. The final tax liability is the value at the root; every other figure on the

return, every audit trail, and every explanation is a view of this graph.
Alternatives considered: (a) Engine produces only the final number, with explanation reconstructed afterwards by re-running with logging enabled; (b) Engine emits a side-channel log during evaluation.
Rationale: Reconstruction-based explanation (a) is fragile — small refactors of the engine silently break the explanation. Logging (b) violates DD-07 by introducing effects. Treating the trace as the *output* of the function makes the engine pure, makes the trace type-checked, and makes "why is this number this number?" a graph traversal rather than a re-computation. It also makes the explanation provably consistent with the figure: they are derived from the same value.
Trade-offs: The `Computation` value is larger than a single number. For P1 this is unproblematic; for P3a, where Monte Carlo produces many computations, we will need to consider whether to retain full traces for every path or only summaries.

9.16 DD-16: Provenance Is a Threaded Value, Not a Sidecar

Date: May 2, 2026
Phase relevance: Cross-cutting; foundational for the audit trail.
Decision: Every domain value that crosses the trust boundary (`Fact`, `ClassifiedFact`, every node in a `Computation`) carries a `Provenance` field as part of its type. Provenance is not a global log, not a metadata sidecar, not a database join. Given any value in the system, the chain of inputs that produced it is reachable directly from the value itself.
Alternatives considered: (a) Centralised event log keyed by correlation ID; (b) hash-based content addressing with provenance stored externally; (c) provenance reconstructed on demand by re-running the pipeline.
Rationale: Tax-figure auditability is non-negotiable. A sidecar log can drift out of sync with the values it claims to describe; a threaded value cannot. The cost is some extra payload on every value and some discipline in propagating provenance through transformations; both are acceptable. This is also the cleanest way to honour the reproducibility-and-auditability principle (§2) without relying on external infrastructure.
Trade-offs: Larger in-memory values; the type system must thread provenance through every transformation, which adds boilerplate that is worth it.

9.17 DD-17: Recommendation Constructible Only Through the Suitability Gate

Date: May 2, 2026
Phase relevance: Defines the architectural commitment that P3b (§8) lives or dies on.
Decision: The `Recommendation` type from DD-10 has no public constructor. The only way to obtain a `Recommendation` value is to pass through the Suitability Engine, whose signature is `suitable :: ClientProfile -> Computation -> Either Unsuitable Recommendation`. The `Unsuitable` branch is itself a structured value carrying the reason the recommendation failed (capacity for loss exceeded, time horizon mismatch, vulnerability flag, missing data, etc.). There is no second constructor, no convenience function, no test-only back door.
Alternatives considered: (a) A `Recommendation` with a public constructor and a separate "please run suitability before emitting" convention enforced by code review; (b) suitability as an after-the-fact check that filters a stream of already-constructed recommendations; (c) suitability checks scattered through the codebase wherever recommendations are generated.
Rationale: Convention-based gating fails under deadline pressure and during refactors. Filtering after-the-fact means a wrongly-typed `Recommendation` can briefly exist in memory, in logs, in error messages, in caches — exactly the surfaces a regulator examines after a complaint. Scattered checks are unauditable. A single typed gate makes "did suitability run?" equivalent to

"does this value exist?", which is the strongest invariant the type system can offer and the easiest claim to defend in a Section 166 review.
Trade-offs: The `ClientProfile` must be rich enough to discharge every COBS 9 suitability concern; building it is most of the 3b software effort. The gate adds a step that cannot be skipped even in internal tooling, which is the point.

9.18 DD-18: Python Is for the Ecosystem, Not for the Adapter Layer

Date: May 2, 2026
Phase relevance: Cross-cutting; refines DD-12 by naming the criterion for what stays in Python and what migrates inward.
Decision: A piece of code stays in Python if and only if it depends materially on the Python AI/ML ecosystem (LLM SDKs, OCR libraries, document-parsing libraries, ML frameworks). Pure I/O code without such a dependency — HMRC Making Tax Digital API clients, persistence and database access, parsing of well-defined file formats, JSON-schema validation at the trust boundary — is a legitimate inward-migration target on the same ratchet (DD-13) as computation code.
Alternatives considered: (a) "All adapter work stays in Python" (the original DD-12 framing — rejected: confuses ecosystem with layer); (b) "Imperative shell stays in Python, functional core in Haskell" (rejected: `IO` in Haskell is not an inferior `IO`; the real reason for Python is library access, and naming it correctly is what lets us reason about migration honestly); (c) "Migrate everything to Haskell eventually" (rejected: the Python AI/ML ecosystem is genuinely irreplaceable for OCR, LLM calls, document classification in the foreseeable future; pretending otherwise leads to worse adapters built on weaker libraries).
Rationale: Haskell's `IO` is perfectly capable of any adapter work that does not need Python's libraries, and a typed adapter is in fact *stronger* at the trust boundary than a Python one — there is no Pydantic-to-Haskell deserialisation step, no second validation, and refactors across the boundary are caught by the compiler. The original DD-12 framing of "Python permanent in adapter and interface layers" was an over-simplification: it conflated "messy I/O" (which is no obstacle to Haskell) with "ML-library access" (which genuinely is, today). DD-18 names the criterion correctly so that subsequent migration decisions can be made on the right basis.
Practical consequences:

- Inward-migration candidates beyond computation: HMRC MTD client; persistence layer; schema validation at the trust boundary; structured-PDF and CSV parsers.
- Permanent Python residents: OCR pipelines; LLM-mediated classification and extraction; layout-aware document parsers; anything that calls into the Python ML ecosystem.
- As more adapter work migrates inward, the surface that needs DD-14's wire protocol shrinks. DD-14 remains correct for whatever stays in Python, but should not be treated as load-bearing on the parts that have crossed the boundary.

Trade-offs: More nuance in deciding what migrates than a hard layer rule. But the hard rule was wrong, so the trade is necessary.

10 Open Questions

- Which AI provider for classification: self-hosted, Anthropic API, or both with a fallback chain?
- Local-first vs cloud for receipt storage (privacy implications)?

- How much of HMRC's published guidance can we encode declaratively vs as imperative checks?
- Versioning strategy when tax rules change mid-project.
- P2: how do we surface scenario *combinations* without overwhelming the user with a combinatorial explosion of options? Ranking, pruning, and a sensible default presentation are open.
- Strategic direction after P2: 3a (consumer financial-planning product), 3b (horizontal regulatory-compliance platform), both, or neither? These are different businesses with different customers, sales motions, and unit economics. A firm at this scale cannot pursue both well at once; the choice has to be made.
- P3b: which regime is the second one (after HMRC tax)? The second regime is the one that forces the right generalisation; the candidates listed in §8 have different commercial and architectural fits. AML/KYC and FATCA/CRS are the closest to the P1 kernel; FCA personal advice is the most architecturally specified (DD-17); GDPR provenance is the most directly served by the existing kernel.
- P3b: market sizing. The horizontal-RegTech thesis should be backed by hard numbers (TAM, SAM by regime, competitor maturity) before significant 3b investment, not after. This is a strategic document not produced here.
- P3a: which distributional assumptions do we use for return modelling (historical bootstrap, parametric, regulator-supplied scenarios)? Whichever we pick must be documented and challengeable by the user.
- DD-01: when do we commit to Haskell vs Lean? The trigger should be the moment the first non-trivial inner-core module is needed — probably the rule evaluator at the end of P1 or start of P2.
- DD-12: which module migrates inward first? Candidate: `Money` plus its arithmetic, since it is small, ubiquitous, and the unit economics of the migration cost are well-understood.
- DD-14: at what point does subprocess-and-JSON cease to be adequate? Define a latency budget per session for P3a and revisit when we approach it.
- P1 Rule Library (§5): what is the source format for the rule library? Hand-written code in the inner-core language, a declarative DSL, or YAML curated by a tax adviser? Each has trade-offs in expressivity, auditability by non-programmers, and confidence that the encoding matches HMRC's intent.
- DD-18: which non-AI adapter component is the right first inward-migration target after `Money`? Strong candidates: a typed HMRC Making Tax Digital API client (well-defined external contract, tax-domain, ratchets nicely with the rule library); persistence (a typed `Store Fact` interface buys end-to-end typing on the audit trail). Weaker: file-format parsers, which often drift back toward the AI/OCR pipeline anyway.

11 Change Log

- May 2, 2026— Initial document. DD-01 through DD-06 recorded.
- May 2, 2026— Added phased roadmap (P1 filing, P2 what-if, P3 planning). Added DD-07 (pure replayable computation), DD-08 (scenarios as input transformations), DD-09 (time as a first-class dimension), DD-10 (computation/guidance/advice separation, FCA boundary), DD-11 (seeded Monte Carlo for projections).

- May 2, 2026— Implementation strategy revised. DD-01 reframed as Haskell-or-Lean (deferred); architectural overview rewritten as concentric layers with Python outer / typed inner core. Added DD-12 (Python-first, migrate inward), DD-13 (ratchet discipline), DD-14 (Python↔inner-core boundary protocol). Appendix restructured to track Python and inner-core modules separately.
- May 2, 2026— Document restructured. Added §Philosophy (strong typing, compositional development, correct by construction, functional core / imperative shell, reproducibility and auditability). Each phase promoted to its own top-level section (§4, §6, §7) with goal, scope, architectural focus, success criterion, and risks.
- May 2, 2026— Added §5 (Phase 1 Component Design): pipeline, components (Ingestion, Fact Builder, Classifier, Rule Library, Tax Engine, Return Assembler, Explanation Renderer, Reconciliation), and cross-cutting concerns (provenance, versioning, persistence). Added DD-15 (computation as a trace) and DD-16 (provenance as a threaded value).
- May 2, 2026— Document retitled from *AI Tax System* to *From AI Tax to AI Financial Planning*, reflecting the trajectory across the three phases. Introduction reframed accordingly.
- May 2, 2026— Document retitled to *Starting Simple: AI Tax*.
- May 2, 2026— Phase 3 split into Phase 3a (Financial Planning Advice, guidance product) and Phase 3b (Regulatory Compliance, the path to FCA-authorised personal advice). Added DD-17 (Recommendation constructible only through the Suitability Gate). Cross-references in DDs and Open Questions updated to distinguish 3a-specific concerns (Monte Carlo, time, latency) from 3b-specific concerns (authorisation, suitability, supervision).
- May 2, 2026— Architectural overview refined: the Python/inner-core boundary is criterion-driven, not layer-driven. Added DD-18 (Python is for the ecosystem, not for the adapter layer). DD-12 end-state description softened accordingly. Open Questions extended with the next-adapter-to-migrate question.
- May 2, 2026— Strategic posture sharpened. P1 architectural focus rewritten to make explicit that the inner core grows as far as possible and appropriate from P1 onwards, not in some later phase. DD-12 rationale sharpened in the same direction: migration begins from day one, not merely planned.
- May 2, 2026— Phase 3b reframed from "crossing into FCA-regulated personal advice" to "regulatory compliance as a horizontal product". The business thesis is that the architecture and team capabilities built in P1/P2 generalise to any regulator that demands rules, evidence, audit, and retention — a multi-£bn market. FCA personal advice retained as the most architecturally detailed first regime. Added candidate regimes (AML/KYC, GDPR, ESG, FATCA/CRS, electronic records, Consumer Duty). Open Questions reworked accordingly.

A Module Reference

This appendix documents every source file in the project. Because the system spans two languages (DD-12), the appendix is split: §A.1 covers the Python codebase (Adapter and Interface layers, plus any not-yet-migrated computation), and §A.2 covers the trusted inner core (Haskell or Lean, see DD-01).

When a Python module is migrated inward (DD-12, DD-13), its entry moves from §A.1 to §A.2, with a back-reference noting the migration date. This appendix is therefore a living record of the migration's progress.

Entry template (Python)

Module: `package.module_name`
Purpose: one-sentence description.
Important functions:

- `function_name(...) -> T` — what it does and which invariants it preserves (note: in Python these are checked at runtime via Pydantic / asserts, not statically).

Migration status: *In Python / Migrated to inner core on YYYY-MM-DD.*

Entry template (inner core)

Module: `Module.Name`
Purpose: one-sentence description.
Important functions:

- `functionName :: Type` — what it does and which invariants it preserves at compile time.

Origin: *Native to core / Migrated from Python module `X` on YYYY-MM-DD.*

A.1 Python Modules

No Python modules have been written yet. Entries will be added here as files are created.

A.2 Inner-Core Modules (Haskell or Lean)

No inner-core modules have been written yet. The first module is expected to be `Tax.Core.Money` (or its Lean equivalent), once DD-01 is resolved.